E-Portfolio

Module: Cloud Operations and Management

Introduction

According to NIST, cloud computing allows users to access computing resources, such as storage, servers and applications, over a network whenever they need them, without having to own or manage the physical infrastructure (Mell and Grance, 2011). These resources are shared between many clients and can be obtained within minutes and released just as quickly, with minimal management effort.

The three service models differ in how responsibilities are distributed between the provider and the client. In Infrastructure as a Service (IaaS), the client rents infrastructure such as virtual machines, storage and networking, and remains responsible for the operating system. In Platform as a Service (PaaS), the provider manages the operating system and the runtime, while the client manages their own code and data. In Software as a Service (SaaS), the client only manages their own data and access.

NIST defines four deployment models: public, private, community and hybrid cloud, which differ in how the infrastructure is owned, managed and accessed (Mell and Grance, 2011).

During this module, I worked on hybrid cloud design through two projects: a team report evaluating Azure Resource Manager (Unit 6) and an individual cloud strategy for Medevac Air Rescue, a hypothetical air ambulance company (Unit 11). In practice, I used both a public cloud (Azure) and a local virtual machine.

Looking back, I focused more on reading than on practising. I now try to practise more, for instance through the disaster recovery exercise in Unit 8 and the use of terraform validate in the Medevac project.

This e-portfolio first reflects on five key topics from the module, then describes the skills I developed, applies cloud principles to the air ambulance sector, and sets out my future learning goals.

Weekly Reflections

- Unit 1 Choosing a Cloud Provider:

In the Unit 1 formative discussion, I compared AWS and Google Cloud on price, availability zones, instance types and AI/ML capabilities (Rachita et al., 2025).

The feedback revealed two blind spots. First, a peer raised vendor lock-in, which I had not considered. Depending on a single vendor's infrastructure is risky because leaving is difficult and expensive: if prices rise, there may be no alternative, and some services are hard to migrate (Opara-Martins, Sahandi and Tian, 2016). Second, the tutor noted that total cost depends on region, data transfer and utilisation. I only understood this in Unit 4: I did not realise my virtual machines were still running and kept creating new ones, which exhausted my quota and my credit.

Cost depends on usage, not on the listed price. I also relied on a single source and did not verify figures that change quickly.

Since then, I have treated vendor lock-in as a design criterion: it appears in the Unit 6 team report and in the Medevac risk matrix, where Terraform acts as a safeguard because the code is partly portable.

- Units 4–5 Automation and Cloud Governance:

In Units 4 and 5, I had to write a Bash script that provisions an Azure virtual machine, configures it and runs basic commands automatically. I spent a week on it, convinced that the failures came from my own lack of understanding. The real cause was the platform, not my code. Using az vm list-usage, I found that the virtual machines described above had exhausted my vCPU quota; az account show later revealed that my subscription had been disabled once my credit ran out. When I returned to the exercise weeks later, I could no longer execute the script at all. In organisations, quotas, budgets and subscriptions are managed by administrators as part of cloud governance. My experience showed why: without budget alerts or quota monitoring, a simple oversight can block all technical work.

If I did it again, I would first verify the environment, including subscription status, quota and the VM sizes available in the chosen region, and delete the resource group after each exercise.

- Unit 6 Infrastructure as Code and Vendor Lock-in:

In Unit 6, our team of three evaluated Azure Resource Manager (ARM) and designed a hybrid infrastructure around it; I wrote the overview section. The team was well organised, each member taking one section, and a colleague's professional cloud experience helped us link theory to practice. The report received 76%.

The tutor noted that our analysis could be deepened by comparing ARM with alternative frameworks. We had not considered any. Yet ARM only works on Azure: writing infrastructure as ARM templates is itself a form of vendor lock-in, the risk I had overlooked in Unit 1. Evaluating a tool in isolation limited our critical analysis.

In the Medevac project (Unit 11), I chose Terraform after it was presented in the lecturecasts. With hindsight, the feedback confirmed this choice: Terraform reduces vendor lock-in, since one language and workflow manage both the HDS and public cloud environments, although provider-specific resources would still need rewriting to change provider.

- Unit 8 Disaster Recovery:

In Unit 8, I designed a disaster recovery plan for a ransomware scenario, simulated by deleting a production directory. As no VM size was available in my Azure subscription, I implemented it locally on Kali Linux with Restic. The measured RPO was 68 seconds and the RTO 5 seconds, but these values cannot be generalised: only 39 bytes were restored, locally, with no network transfer.

The most valuable lesson came from a mistake. My first test backed up an old Restic repository instead of the production data and was still reported as a complete success. A backup is only proven when the restored content is checked. My peer

and tutor pushed this further: my verification with cat was manual and limited, and a systematic integrity check, such as checksums or restic check, would be stronger. The tutor also noted that recovery roles and communication procedures were missing.

I applied this lesson in the Medevac project, where restoration tests are scheduled, automated and verified. However, recovery roles and communication procedures are still absent there, a gap I would now address.

- Unit 11 AI in Cloud Operations:

Unit 11 explored how AI enhances cloud operations, particularly through automation. In the Medevac project, I proposed an anomaly detection service that learns each account's usual patterns and flags unusual access to patient records, since the company has no SOC and no one reviews logs at night.

However, I chose prioritised alerts and a human decision rather than automatic action. My ICU experience shaped this choice: a monitor can alarm simply because an ECG electrode is misplaced. When false alarms are frequent, clinicians start to ignore them, a phenomenon known as alarm fatigue (Sendelbach and Funk, 2013). AI-based detection carries the same risk: a false positive could lock a medical account during a repatriation, and repeated false alerts could eventually be ignored.

This changed how I see AI for a future SOC role: it is a support for decisions, not a decision-maker, and it still needs human oversight.

Skills Development

Restic was the tool I used most practically. It encrypts data client-side and creates snapshots on demand. In Unit 8, the VirtualBox shared folder (vboxsf) could not delete Restic's lock files, which blocked later operations. I worked around it with --no-lock, which runs certain commands without locking the repository. This solved the problem but removed a safety mechanism, and in a real incident the delay would have increased the RTO.

With the Azure CLI, I learned to diagnose the environment before questioning my code: az vm list-usage to check quotas, az account show to check the subscription status. I also learned Azure's naming rules: storage account names must be lowercase and globally unique. My Bash skills developed through errors: when I kept the angle brackets of a placeholder such as <name>, Bash interpreted them as a file redirection. I designed a provisioning script using variables and a final resource group deletion, but I could not execute it once my subscription was disabled.

For Terraform, in the Medevac project I adapted the HashiCorp documentation for the AzureRM provider, adding geo-replication, soft delete and lifecycle rules to meet retention requirements. terraform validate checked the syntax and configuration without deploying anything, at no cost. However, validation does not prove that a deployment would succeed, so my Terraform skills remain theoretical until applied.

Overall, my strongest skill is now diagnosis rather than deployment.

Application to Industry

Medical air repatriation places specific demands on cloud computing: services must be available 24/7, since a mission can be triggered at any hour; connectivity is limited during flights; and health data is subject to strict regulation. In the Medevac project, I applied cloud principles to these needs through a hybrid design, with health data hosted by an HDS-certified provider and other workloads on the public cloud.

The choice I would defend most is edge computing on board. In 2026, in-flight connectivity still relies on satellite links that are intermittent, slow and billed by volume, so displaying vital signs cannot depend on a cloud response. Processing data locally and synchronising on landing answers a real operational constraint rather than following a trend. The hybrid allocation is equally justified, as it is imposed by regulation, not by technical preference.

However, a critical evaluation reveals three weaknesses. First, the strategy relies on Terraform in a company with no IT team, so nobody would maintain the code. I would now recommend a small internal IT capability rather than depending entirely on an external provider, although even one employee could absorb a large part of the €30,000 annual budget. Second, I did not cost the solution; I only estimated it vaguely, so its financial feasibility remains unproven. Third, the incident response is incomplete: as my tutor noted in Unit 8, recovery roles and communication procedures are missing. During a ransomware attack, the plan does not specify who restores the systems, who informs the medical team and hospitals, or who notifies the CNIL within 72 hours.

This last gap matters because ransomware is the most pressing threat for this sector, as health organisations remain heavily targeted (ANSSI, 2026). A sound

architecture is not enough if the people responding to an incident do not know their roles.

Future Learning Goals

At the start of the module, my goal was to acquire the foundations of cloud computing. I have achieved this conceptually, but my practical skills remain limited, partly because my Azure environment was unavailable for much of the module. My plan therefore prioritises practice in free local environments, aligned with my target career in digital forensics and incident response (DFIR).

The emerging technologies from Weeks 10 to 12 shape this plan. Serverless computing raises a forensic challenge: functions are short-lived and leave no server to examine, so evidence exists only in logs. AI will increasingly automate detection, which makes understanding its limits essential, as discussed above. Quantum computing threatens current encryption through “harvest now, decrypt later” attacks, a risk for evidence and health records retained for decades (CISA, NSA and NIST, 2023).

My goals are:

1. Before the end of my next module, build a local Docker and Minikube lab and deploy a containerised application, completing the activity I missed in Unit 5.
2. During the following module, practise cloud log analysis for incident response, using Azure activity and sign-in logs, as this is where cloud evidence resides.
3. Before the end of my MSc, assess whether the SC-200 certification fits my SOC and DFIR goals.

References

ANSSI (2026) *Panorama de la cybermenace 2025*. Available at: https://cyber.gouv.fr/nous-connaitre/publications/panoramas-de-la-cybermenace/panorama-de-la-cybermenace-2025/ (Accessed: 1 October 2026).

CISA, NSA and NIST (2023) *Quantum-Readiness: Migration to Post-Quantum Cryptography*. Available at: https://www.cisa.gov/resources-tools/resources/quantum-readiness-migration-post-quantum-cryptography (Accessed: 15 October 2026).

Mell, P. and Grance, T. (2011) *The NIST Definition of Cloud Computing*. NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology. doi: 10.6028/NIST.SP.800-145.

Opara-Martins, J., Sahandi, R. and Tian, F. (2016) ‘Critical analysis of vendor lock-in and its impact on cloud computing migration: a business perspective’, *Journal of Cloud Computing*, 5, article 4. doi: 10.1186/s13677-016-0054-z.

Rachita, Sharma, M., Chopra, S. and Galhotra, B. (2025) ‘Comparative analysis of Amazon Web Services (AWS), Azure, and Google Cloud: core services, market strategy, and pricing models’, in *New Horizons of Science, Technology and Culture*, Vol. 4. 1st edn. BP International, pp. 33–45. doi: 10.9734/bpi/nhstc/v4/6019.

Sendelbach, S. and Funk, M. (2013) ‘Alarm fatigue: a patient safety concern’, *AACN Advanced Critical Care*, 24(4), pp. 378–386. doi: 10.1097/NCI.0b013e3182a903f9.